

Market Regime Detector + Portfolio Optimizer

Hidden Markov Model · Markowitz Optimization · Streamlit Dashboard

Sharpe target	Max drawdown	CAGR target	Build time
1.4+	-18%	14%+	5–6 weeks

1. The idea

Most retail portfolios use static allocations — 60/40, forever. The real insight is that markets cycle through measurable **regimes**: Bull (low volatility, positive drift), Bear (high volatility, negative drift), and Sideways/Volatile. A Hidden Markov Model (HMM) can detect which regime you are currently in, then a Markowitz optimizer shifts your portfolio weights accordingly. This is how quant funds like Bridgewater and AQR actually manage risk.

The project combines three real-world techniques: **unsupervised temporal ML** (HMM), **convex portfolio optimization** (mean-variance / risk parity), and a **walk-forward backtest** with transaction costs. The output is a live Streamlit dashboard where you can see the current regime and recommended allocation.

What makes it stand out

Insight	Why it matters
Markets are non-stationary	HMMs capture this — they model temporal persistence of regimes, unlike K-Means which treats
Regime-aware allocation	Bull: equity tilt (SPY, QQQ). Bear: defensive (TLT, GLD). Sideways: risk parity. Real funds do e
Walk-forward backtest	No lookahead bias. Each day only uses data available up to that point. Recruiters will ask you a
Live dashboard	Deployed on Streamlit Cloud — you can share a live link, not just a notebook.

2. System architecture

1 · Data ingestion

- yfinance — OHLCV for SPY, QQQ, IWM, GLD, TLT, LQD, VNQ, USO (8–12 ETFs, 15+ years)
- FRED API — VIX, 10Y-2Y yield spread, CPI, unemployment rate
- pandas-datareader — Treasury yields, DXY, commodity indices

2 · Feature engineering

- Log returns, rolling vol (20d / 60d), realized vol ratio
- Momentum: 1M, 3M, 6M return z-scores
- Macro: VIX level, yield curve shape, DXY change
- Cross-asset: bond-equity rolling 60d correlation, 50/200 MA ratio

3 · Regime detection — the ML core

- GaussianHMM (hmmlearn) — 3 hidden states, full covariance
- Viterbi decoding for most-likely state sequence
- State persistence filter — min 5-day hold to avoid flip-flopping
- Rolling refit every 252 trading days (expanding window)
- States: Bull (low vol, +drift) · Bear (high vol, -drift) · Sideways/Volatile

4 · Portfolio optimization

- Mean-Variance (PyPortfolioOpt) — max Sharpe per regime
- Ledoit-Wolf covariance shrinkage for stability with 10+ assets
- Risk Parity as bear-regime fallback (equal risk contribution)
- Constraints: long-only, max 30% single asset, min 5% allocation

5 · Backtest + output

- vectorbt walk-forward backtest with 0.1% transaction costs
- quantstats tearsheet: Sharpe, Sortino, max drawdown, CAGR, Calmar
- Benchmark comparison: SPY, 60/40, equal-weight
- Streamlit dashboard: regime timeline, allocation pies, equity curve

3. Plan of action — 6 phases

Week 1

Data pipeline + environment setup

Build the foundation. Everything downstream depends on clean, reliable data.

- Fetch 15+ years of daily OHLCV for 10–12 ETFs via yfinance
- Pull macro indicators from FRED: VIX, 10Y-2Y spread, CPI, unemployment
- Write a DataLoader class with parquet caching (no repeated API calls)
- Handle missing data, survivorship bias, and corporate actions
- EDA notebook: price history, correlations, missing value heatmap

yfinance, fredapi, pandas-datareader, pyarrow, matplotlib

Checkpoint: Clean dataset saved to parquet, EDA notebook complete.

Week 2

Feature engineering

Build the feature store that feeds the HMM. Think like a quant — what signals differentiate regimes?

- Compute log returns, rolling vol (20d/60d), realized vol ratio
- Momentum features: 1M, 3M, 6M return z-scores
- Macro features: VIX level, yield curve shape, DXY change
- Cross-asset: bond-equity correlation (rolling 60d), commodity trend
- Write a FeatureEngineer class; output normalized feature matrix
- Feature importance analysis using mutual information

pandas, numpy, scipy, sklearn.preprocessing, seaborn

Checkpoint: Feature matrix class with unit tests; correlation heatmap.

Week 2–3

HMM regime detection — the ML core

Train a Gaussian HMM on your feature matrix. Validate against known events (2008, 2020, 2022).

- Train GaussianHMM with 3 states (n_components=3, covariance_type="full")
- Viterbi algorithm to decode most-likely state sequence
- State persistence filter — prevent flip-flopping (min 5-day hold)
- Rolling refit: retrain every 252 trading days on expanding window
- Visualize: price chart with regime color bands, transition matrix heatmap
- Tune n_components with BIC/AIC score comparison

hmmlearn, sklearn, plotly, joblib

Checkpoint: Regime chart correctly identifies 2008 bear + March 2020 crash. Show this in your portfolio!

Week 3–4

Portfolio optimizer — regime-aware allocation

For each regime, compute a distinct optimal allocation. Bull = equities. Bear = bonds/gold. Sideways = risk parity.

- Compute regime-conditional expected returns (mean returns during each state)
- Covariance estimation with Ledoit-Wolf shrinkage
- Solve max-Sharpe portfolio per regime using PyPortfolioOpt
- Implement Risk Parity as bear regime fallback
- Constraints: long-only, max 30% single asset, min 5%
- Display regime-specific allocation weights as pie charts

PyPortfolioOpt, cvxpy, sklearn.covariance, plotly

Checkpoint: 3 distinct allocation tables (one per regime) + pie charts.



Week 4–5

Backtesting engine

Walk-forward backtest: only use data available at each point in time. No lookahead bias.

- Implement walk-forward backtest loop using `vectorbt`
- Rebalance when HMM detects regime change (+ 0.1% transaction costs)
- Compute: Sharpe, Sortino, max drawdown, CAGR, Calmar ratio
- Benchmark vs SPY, 60/40, equal-weight portfolio
- Stress test: 2008, 2020, 2022 performance specifically

`vectorbt`, `quantstats`, `pandas`, `plotly`

Checkpoint: Backtest tearsheet showing outperformance vs benchmark.



Week 5–6

Dashboard, deployment + portfolio polish

Ship it. Build the Streamlit app, deploy to Streamlit Cloud (free), write a compelling README.

- Streamlit app: sidebar (asset selection, date range, `n_regimes` slider)
- Page 1 — Regime detector: price chart + colored regime bands
- Page 2 — Portfolio: allocation pies per regime + current recommended weights
- Page 3 — Backtest: equity curve, performance table, drawdown chart
- Deploy to Streamlit Community Cloud (free hosting)
- Write README with problem statement, methodology, results, screenshots

`streamlit`, `plotly`, Streamlit Cloud, GitHub Actions

Checkpoint: Live deployed app + GitHub repo with a stars-worthy README.

4. Full tech stack

DATA yfinance Free daily OHLCV for any ticker, 20+ years of history fredapi Official FRED data — VIX, yield curve, macro indicators pandas-datareader Supplementary economic time series pyarrow Parquet storage — fast I/O, no repeated API calls	ML · REGIME DETECTION hmmlearn GaussianHMM — core regime detection model scikit-learn Preprocessing, scaling, BIC/AIC model selection scipy Statistical tests, optimization routines joblib Model serialization + parallel rolling refit
PORTFOLIO OPTIMIZATION PyPortfolioOpt Max Sharpe, Risk Parity, Black-Litterman ready cvxpy Convex optimizer underlying PyPortfolioOpt sklearn.covariance Ledoit-Wolf shrinkage estimator numpy Matrix ops, portfolio math, return calculations	BACKTESTING vectorbt Fast vectorized backtest with portfolio analytics quantstats One-line tearsheet: Sharpe, Sortino, drawdown pandas Time-series alignment, rebalancing logic
DASHBOARD streamlit App framework — no frontend knowledge needed plotly Interactive charts: regime bands, equity curves, pies Streamlit Cloud Free deployment — share live link in portfolio	DEV + STRUCTURE Python 3.11+ Core language pytest Unit tests for DataLoader and FeatureEngineer Jupyter EDA and regime validation notebooks GitHub Repo + README — your actual portfolio artifact

Suggested folder structure

```
regime_portfolio/  
  
data/ ← raw + processed parquet files  
  
src/ ← DataLoader, FeatureEngineer, HMMModel, Optimizer  
  
notebooks/ ← EDA, regime_validation, backtest_analysis  
  
app/ ← streamlit_app.py + pages/  
  
tests/ ← pytest unit tests  
  
README.md ← your portfolio showpiece
```

5. Interview questions — know these cold

Q	A
How did you prevent lookahead bias?	Rolling refit + walk-forward backtest. Each refit uses only data available up to that date. No future data leaks into any decision.
Why HMM over K-Means clustering?	HMM captures temporal persistence — markets stay in regimes. K-Means assigns each day independently with no memory of yesterday's state. Markets don't randomly flip states.
How do you handle regime uncertainty?	State persistence filter (min 5-day hold before regime change is confirmed) + Risk Parity as a defensive fallback during uncertain/bear regimes.
How do you choose the number of states?	Compare BIC scores for n=2,3,4. Also overlay on price history and validate states match known market events (2008, 2020, 2022 bear).
Why Ledoit-Wolf shrinkage?	Raw sample covariance is unstable with 10+ assets on short windows. Ledoit-Wolf shrinks toward a structured estimator, reducing estimation error and improving out-of-sample optimizer performance.

Recommended ETF universe to start with

Ticker	Name	Role in portfolio
SPY	SPDR S&P 500 ETF	Core US equity exposure
QQQ	Invesco Nasdaq 100	Growth / risk-on tilt
IWM	iShares Russell 2000	Small-cap equity
GLD	SPDR Gold Shares	Safe haven / inflation hedge
TLT	iShares 20+ Year Treasury	Long-duration bonds / bear hedge
LQD	iShares Investment Grade Corp	Credit / yield
VNQ	Vanguard Real Estate ETF	Real estate / inflation
USO	United States Oil Fund	Commodity / macro signal

Next step: say "let's start Phase 1" and we'll write the DataLoader class together — yfinance + FRED integration, parquet caching, and the full data pipeline. Line by line, as a team.